

06. Building & Training a Modern LLM — From Scratch

Complete Beginner-Friendly Notes

Topics covered: RMSNorm, RoPE, GQA, SwiGLU, Training Loop, Text Generation

The Big Picture — What Are We Building?

Imagine you're building a robot that reads Shakespeare and then tries to write like Shakespeare. That's exactly what this code does. We're building a **mini language model** — the same kind of technology behind ChatGPT, Claude, and other AI chatbots — just a much smaller version.

The model learns one simple task: **given some text, predict the next character**. That's it. But from this single task, it learns spelling, grammar, character names, and even dramatic structure.

Think of it like this: if someone showed you "To be or not to b..." — you'd instantly know the next letter is "e". Our model learns to do this, but for ALL of Shakespeare's text.

The architecture we're building uses four modern upgrades that real production LLMs like LLaMA and Mistral use: RMSNorm (simpler normalization), RoPE (smarter position encoding), GQA (memory-efficient attention), and SwiGLU (better feed-forward networks).

Section 0: Setup & Data

0.1 — Importing Libraries

python

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math, time, os
device = "cuda" if torch.cuda.is_available() else "cpu"
```

What's happening: We import PyTorch (our deep learning framework) and check if a GPU is available. Training on a GPU is 10-50x faster than on CPU. Think of the GPU as a factory with thousands of workers doing math in parallel, while a CPU is like one very smart person doing math alone.

0.2 — Loading Shakespeare

We download the "tiny Shakespeare" dataset — about 1.1 million characters of Shakespeare's plays. This is our training data. The model will read this text and learn patterns from it.

0.3 — Character Tokenizer

python

```
chars = sorted(set(text))          # All unique characters
vocab_size = len(chars)            # 65 total

char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for i, c in enumerate(chars)}

def encode(s): return [char_to_idx[c] for c in s]
def decode(ids): return "".join([idx_to_char[i] for i in ids])
```

The key idea: Computers can't read letters — they only understand numbers. So we need to convert every character into a number.

We find all unique characters in Shakespeare (65 total — letters, spaces, punctuation, newlines) and assign each one a number.

Example: "Hello" → [20, 43, 50, 50, 53]

Each character gets a unique ID. `encode()` converts text to numbers, `decode()` converts numbers back to text. This is the simplest possible tokenizer. Real LLMs like GPT use **BPE (Byte Pair Encoding)** which works with word pieces instead of individual characters, but character-level is easier to understand.

0.4 — Train/Val Split

python

```
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data))
train_data = data[:n]      # 90% for training
val_data = data[n:]        # 10% for testing
```

We split the data into two parts: 90% for training (what the model learns from) and 10% for validation (to check if the model actually learned, or just memorized). This is like studying from a textbook (training) and then taking a quiz on unseen questions (validation).

0.5 — get_batch() — Grabbing Training Data

python

```
def get_batch(split, batch_size, context_length):
    d = train_data if split == "train" else val_data
    ix = torch.randint(len(d) - context_length, (batch_size,))
    x = torch.stack([d[i:i+context_length] for i in ix])
    y = torch.stack([d[i+1:i+context_length+1] for i in ix])
    return x.to(device), y.to(device)
```

What this does: Picks random chunks of text from our dataset. For each chunk:

- `x` = the input (e.g., "To be o")
- `y` = the target, shifted right by 1 character (e.g., "o be or")

Why shift by 1? Because our model's job is to predict the NEXT character. So at every position, the correct answer is whatever comes one step ahead.

Here's how one sequence of "To be," gives us 5 training examples:

Position	Model Sees	Should Predict
0	T	o
1	To	(space)
2	To_	b
3	To_b	e
4	To_be	,

One sequence of length N gives us N training examples! This is very efficient.

Section 1: RMSNorm (Root Mean Square Normalization)

Why Do We Need Normalization?

Imagine you're baking a cake and your ingredients are wildly different sizes — 1 gram of salt and 10,000 grams of flour. It would be hard to adjust the recipe because tiny changes to the flour barely matter, while tiny changes to the salt matter a lot. Normalization is like converting everything to "cups" so all ingredients are on the same scale.

In neural networks, without normalization, some numbers in our vectors become very large and others very small. This makes training unstable — the model struggles to learn because the gradients (learning signals) become either too big or too small.

LayerNorm vs RMSNorm

LayerNorm (the old way) does 4 operations:

1. Calculate the mean of all values
2. Subtract the mean (center to zero)
3. Divide by standard deviation (scale to unit variance)
4. Apply learnable scale AND shift parameters

RMSNorm (the new way) does only 2 operations:

1. Divide by the Root Mean Square (RMS)
2. Apply learnable scale parameter only

RMSNorm skips the mean subtraction and the shift. Research showed these extra steps don't actually help much, so why waste compute? It's simpler, faster, and works just as well. Used in LLaMA, Mistral, and Gemma.

RMSNorm — The Code

python

```
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x):
        rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return (x / rms) * self.weight
```

Step by step with a real example:

Take the vector `[3, -1, 2, -4]`:

1. **Square each value:** `[9, 1, 4, 16]`
2. **Take the mean:** `(9+1+4+16)/4 = 7.5`
3. **Take the square root:** `√7.5 ≈ 2.74` — this is the RMS
4. **Divide each original value by RMS:** `[3/2.74, -1/2.74, 2/2.74, -4/2.74] = [1.09, -0.36, 0.73, -1.46]`
5. **Multiply by learnable weight:** The model learns the best scale for each dimension

The `eps = 1e-6` (a tiny number: 0.000001) is added before the square root to prevent division by zero if all values happen to be zero.

Dropout — Regularization

python

```
DROPOUT = 0.2
```

Dropout is used alongside normalization throughout the model. During training, it randomly "turns off" 20% of the neurons (sets their output to zero). This forces the network to not rely too heavily on any single neuron.

Think of it like a sports team where the coach randomly benches different players during practice. The team has to learn to play well regardless of who's on the bench. During the actual game (inference), everyone plays.

Section 2: RoPE (Rotary Positional Embeddings)

Why Does Position Matter?

The sentence "The dog bit the man" means something completely different from "The man bit the dog." The words are the same — only the ORDER changed. So our model needs to know WHERE each token is in the sequence.

The Old Way (Sinusoidal)

The original Transformer added a fixed pattern of numbers to each token's embedding based on its absolute position. Position 0 always gets the same pattern, position 1 gets another, and so on. The problem? It only captures ABSOLUTE position — it knows "this word is at position 5" but doesn't naturally understand "these two words are 3 positions apart."

The New Way (RoPE)

RoPE takes a completely different approach: instead of ADDING position information, it ROTATES the Query and Key vectors based on their position.

The Clock Analogy: Think of a clock. The angle between 3 o'clock and 5 o'clock is the same as the angle between 7 o'clock and 9 o'clock — both are a gap of 2 hours. The absolute numbers don't matter; only the gap matters.

RoPE works the same way. After rotation, when we compute the dot product of Q and K (which determines how much attention one token pays to another), the result depends only on the RELATIVE distance between the two positions, not their absolute positions.

So "Position 3 attending to Position 5" (gap = 2) gives the same attention pattern as "Position 7 attending to Position 9" (gap = 2).

RoPE — Precomputing Frequencies

python

```
def precompute_rope_freqs(head_dim, max_seq_len, base=10000.0):
    freqs = 1.0 / (base ** (torch.arange(0, head_dim, 2).float() / head_dim))
    positions = torch.arange(max_seq_len).float()
    angles = torch.outer(positions, freqs)
    return torch.cos(angles), torch.sin(angles)
```

Each pair of dimensions gets a different rotation speed (frequency):

- **Low-numbered dimensions** rotate FAST → they capture SHORT-range patterns (like nearby letters)
- **High-numbered dimensions** rotate SLOW → they capture LONG-range patterns (like the theme of a paragraph)

This is precomputed once and reused every time the model processes data — no need to recalculate.

RoPE — Applying the Rotation

python

```
def apply_rope(x, cos, sin):
    x1 = x[..., ::2]    # even dimensions (0, 2, 4, ...)
    x2 = x[..., 1::2]   # odd dimensions (1, 3, 5, ...)
    out1 = x1 * cos - x2 * sin
    out2 = x1 * sin + x2 * cos
    return torch.stack([out1, out2], dim=-1).flatten(-2)
```

The vector is split into pairs: (d0, d1), (d2, d3), (d4, d5)... Each pair is rotated using the standard 2D rotation formula from math:

```
new_x = x * cos(θ) - y * sin(θ)
new_y = x * sin(θ) + y * cos(θ)
```

The angle θ depends on the token's position. This is applied to Q and K only — NOT to V (Value). The Value vectors carry the actual information content, and we don't want to distort that with position encoding.

Section 3: GQA (Grouped Query Attention)

Quick Refresher: What is Attention?

Attention is how a model decides "which other words should I pay attention to when processing this word?" For the sentence "The cat sat on the mat because it was tired," when processing "it," the model needs to attend strongly to "cat" to know what "it" refers to.

In standard **Multi-Head Attention (MHA)**, we have three matrices for each head:

- **Q (Query):** "What am I looking for?"
- **K (Key):** "What do I contain?"
- **V (Value):** "What information do I have?"

Each head has its own Q, K, and V — so 8 heads means 8 Q matrices, 8 K matrices, and 8 V matrices.

The Problem GQA Solves

During inference (when the model generates text), it needs to store all the K and V values from previous tokens — this is called the **KV cache**. With 8 heads, you're

storing 8 sets of K and 8 sets of V. For long sequences and large models, this KV cache becomes HUGE and eats up GPU memory.

GQA: Share KV Heads

GQA's solution is simple: use fewer KV heads than Q heads, and let groups of Q heads SHARE the same KV pair.

In our model:

- 8 Query heads (each head asks its own unique question)
- Only 2 KV heads (shared among groups of Q heads)
- Q heads 0-3 all share KV head 0
- Q heads 4-7 all share KV head 1

This means the KV cache is **4× smaller** than standard MHA. The research finding is that the different Q heads in a group can share the same K and V without losing quality — the queries are doing the differentiation.

GQA — The Projections

python

```
self.q_proj = nn.Linear(d_model, n_heads * head_dim)      # 256 → 256 (8 × 32)
self.k_proj = nn.Linear(d_model, n_kv_heads * head_dim)   # 256 → 64 (2 × 32) ← 4× smaller!
self.v_proj = nn.Linear(d_model, n_kv_heads * head_dim)   # 256 → 64 (2 × 32) ← 4× smaller!
self.o_proj = nn.Linear(n_heads * head_dim, d_model)      # 256 → 256
```

Notice how K and V projections output 64 dimensions (2 heads × 32 dim each) instead of 256. That's where the memory savings come from.

repeat_kv() — Expanding KV Heads

python

```
def repeat_kv(x, n_rep):
    if n_rep == 1: return x
    b, n_kv, seq, hd = x.shape
    return (x[:, :, None, :, :]
            .expand(b, n_kv, n_rep, seq, hd)
            .reshape(b, n_kv * n_rep, seq, hd))
```

Before computing attention scores, we need Q and K to have the same number of heads. This function copies each KV head 4 times so they match the 8 Q heads.

The `expand()` function is clever — it's memory-efficient and doesn't actually copy data, just creates virtual copies.

The Attention Computation

python

```
scale = 1.0 / math.sqrt(self.head_dim) # 1/√32
scores = (q @ k.transpose(-2, -1)) * scale # [b, 8, seq, seq]
```

1. **$Q \times K^T$** : Compute dot product of every Query with every Key. This gives us an "attention score" matrix where each entry tells us how much token i should attend to token j .
2. **Scale by $1/\sqrt{d}$** : Without scaling, the dot products grow large as the dimension grows, pushing softmax into regions with tiny gradients (it saturates). Dividing by $\sqrt{32}$ keeps the numbers in a range where softmax works well.

The Causal Mask

python

```
mask = torch.triu(torch.ones(seq, seq), diagonal=1).bool()
scores = scores.masked_fill(mask, float("-inf"))
```

This is what makes the model "autoregressive" — each token can only look at itself and previous tokens, never future tokens. We fill future positions with negative infinity, which becomes 0 after softmax.

Think of it like a classroom test where you can only see answers from questions you've already answered, not the ones ahead.

	T0	T1	T2	T3
T0	can see	blocked	blocked	blocked
T1	can see	can see	blocked	blocked
T2	can see	can see	can see	blocked
T3	can see	can see	can see	can see

1. **Softmax**: Converts raw scores into probabilities (0 to 1, summing to 1). The $-\infty$ entries become exactly 0.
2. **Weights $\times V$** : The final attention output is a weighted sum of Value vectors, where the weights come from softmax.

Merging Heads Back

python

```
out = out.transpose(1, 2).contiguous().view(b, seq, -1)
return self.o_proj(out)
```

After attention, we concatenate all 8 heads' outputs back into one vector `[b, seq, 256]` and apply a final linear projection. The `.contiguous()` is a technical requirement — after transpose, memory layout may not be contiguous, and `.view()` needs contiguous memory.

Section 4: SwiGLU (Gated Feed-Forward Network)

What's a Feed-Forward Network (FFN)?

After attention mixes information between tokens, each token is processed individually through a feed-forward network. Think of attention as a group meeting where everyone shares info, and the FFN as individual thinking time where each person processes what they heard.

Old FFN vs SwiGLU

Old FFN: One path — expand from 256 to 680, apply ReLU activation, contract back to 256. Simple, but the activation is just a binary gate: positive values pass through, negative values become zero.

SwiGLU: Two parallel paths, then multiply them together.

python

```
class SwiGLU(nn.Module):
    def __init__(self, d_model, hidden_dim):
        self.w_gate = nn.Linear(d_model, hidden_dim) # 256 → 680
        self.w_up = nn.Linear(d_model, hidden_dim) # 256 → 680
        self.w_down = nn.Linear(hidden_dim, d_model) # 680 → 256

    def forward(self, x):
        gate = F.silu(self.w_gate(x)) # Gate path: decides what flows through
        up = self.w_up(x) # Value path: carries the actual info
        return self.w_down(gate * up) # Multiply gate × value, then project back
```

How it works:

1. **Gate path** (`w_gate` + SiLU activation): This learns WHICH dimensions are important and should be kept. $\text{SiLU}(x) = x \times \text{sigmoid}(x)$ — it's a smooth version

of ReLU that allows small negative values through instead of hard-zeroing them.

2. **Value path** (`w_up`, no activation): This carries the raw information without any gating.
3. **Multiply**: `gate × up` — the gate controls how much of each dimension flows through. If the gate outputs 0 for a dimension, that information is blocked. If it outputs 1, it passes fully.
4. **Project back** (`w_down`): Compress from 680 dimensions back to 256.

Think of it like a bouncer at a club. The gate path is the bouncer deciding who gets in (and how enthusiastically), while the value path is the line of people waiting. The multiply step is the bouncer actually letting people through.

Section 5: Putting It All Together

TransformerBlock — One Layer

python

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, n_kv_heads, ffn_hidden_dim):
        self.attn_norm = RMSNorm(d_model)
        self.attention = GrouperQueryAttention(d_model, n_heads, n_kv_heads)
        self.ffn_norm = RMSNorm(d_model)
        self.ffn = SwiGLU(d_model, ffn_hidden_dim)

    def forward(self, x, rope_cos, rope_sin):
        x = x + self.attention(self.attn_norm(x), rope_cos, rope_sin)
        x = x + self.ffn(self.ffn_norm(x))
        return x
```

Each transformer block has two sub-layers, and each follows the same pattern:

Pre-Norm + Sub-layer + Residual Connection

```
x —————
↳ RMSNorm → GQA Attention ————— + (add) → x'

x' —————
↳ RMSNorm → SwiGLU FFN ————— + (add) → out
```

Three important concepts here:

1. **Pre-norm:** We normalize BEFORE each sub-layer, not after. This is more stable during training.
2. **Residual connection** (the `x + ...` part): Instead of completely transforming x , we ADD the sub-layer's output to the original x . The sub-layer only needs to learn the "change" or "update" to x , not rebuild it from scratch. This is like editing a document — it's easier to write corrections than to rewrite the whole thing.
3. **Two sub-layers serve different purposes:**
 - **Attention:** Mixes information BETWEEN tokens ("what should I pay attention to?")
 - **FFN:** Processes each token INDIVIDUALLY ("what do I do with this information?")

MiniLLM — The Full Model

python

```
class MiniLLM(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, ...):
        self.token_emb = nn.Embedding(vocab_size, d_model) # 65 → 256
        self.layers = nn.ModuleList([
            TransformerBlock(...) for _ in range(n_layers) # ×4
        ])
        self.final_norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size) # 256 → 65

        # Weight tying!
        self.lm_head.weight = self.token_emb.weight
```

Key design decisions:

1. **No positional embedding layer:** Unlike the original Transformer, we don't add position information in the embedding. RoPE handles position inside the attention layer instead.
2. **Weight tying:** The embedding layer (that converts token IDs to vectors) and the output head (that converts vectors back to token predictions) SHARE the same weights. This saves parameters and makes sense intuitively — the "meaning" of a token should be consistent whether you're reading it or predicting it.

3. **4 transformer blocks:** Each block independently learns different patterns. Lower layers tend to learn simple patterns (letter combinations), higher layers learn more complex ones (word structure, grammar).

Forward Pass

python

```
def forward(self, idx, targets=None):
    x = self.token_emb(idx)          # [b, seq, 256]
    for layer in self.layers:        # 4 transformer blocks
        x = layer(x, self.rope_cos, self.rope_sin)
    x = self.final_norm(x)           # Final RMSNorm
    logits = self.lm_head(x)         # [b, seq, 65]

    if targets is not None:
        loss = F.cross_entropy(logits.view(-1, 65), targets.view(-1))
    return logits, loss
```

The data flows: **Token IDs** → **Embedding** → **4 Transformer Blocks** → **Final Norm** → **Output Head** → **65 scores per position**

Each position outputs 65 "logits" (raw scores) — one for each possible next character. The highest score is the model's best guess.

Cross-entropy loss measures how wrong the predictions are. If the model assigns high probability to the correct next character, loss is low. If it assigns high probability to wrong characters, loss is high.

Model Configuration

Parameter	Value	Meaning
vocab_size	65	Characters in Shakespeare
d_model	256	Embedding dimension
n_layers	4	Transformer blocks
n_heads	8	Query heads
n_kv_heads	2	KV heads (GQA 4:1 ratio)
ffn_hidden_dim	680	SwiGLU hidden size
max_seq_len	256	Context window

Sanity check: The initial loss should be approximately $\ln(65) \approx 4.17$. Why? If the model is randomly guessing among 65 characters, each character has probability

1/65. Cross-entropy of a uniform distribution over 65 classes = $\ln(65) \approx 4.17$. If the initial loss is close to this, the model is wired correctly.

Section 6: Training

Training Hyperparameters

python

```
BATCH_SIZE = 64          # Process 64 sequences at once
CONTEXT_LEN = 256        # Each sequence is 256 characters
LEARNING_RATE = 3e-4     # 0.0003 – standard for transformers
MAX_STEPS = 3000         # Total training iterations
```

With batch size 64 and context length 256, the model sees $64 \times 256 = 16,384$ characters per training step.

The Training Loop

python

```
for step in range(MAX_STEPS):
    xb, yb = get_batch("train", BATCH_SIZE, CONTEXT_LEN) # ① Get data
    logits, loss = model(xb, yb)                        # ② Forward pass
    optimizer.zero_grad()                               # ③ Clear old gradients
    loss.backward()                                     # ④ Compute gradients
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # ⑤ Clip gradients
    optimizer.step()                                    # ⑥ Update weights
```

This is the same 6-step pattern used in every neural network:

1. **Get data:** Grab a random batch of training sequences
2. **Forward pass:** Feed data through the model, get predictions and loss
3. **Zero gradients:** Clear gradients from the previous step (PyTorch accumulates them by default)
4. **Backward pass:** Compute how each weight should change to reduce the loss (backpropagation)
5. **Clip gradients:** If any gradient is too large (norm > 1.0), scale it down. This prevents "exploding gradients" — a common problem in transformers where gradients can become astronomically large
6. **Update weights:** Apply the gradients to adjust the model's weights

AdamW optimizer is the standard choice for transformers. It's Adam (adaptive learning rate per parameter) plus weight decay (gentle push toward smaller weights to prevent overfitting).

estimate_loss() — Checking How We're Doing

python

```
@torch.no_grad()
def estimate_loss():
    model.eval()          # Turn OFF dropout
    # Average loss over 20 batches for each split
    ...
    model.train()         # Turn dropout back ON
```

Every 250 steps, we pause training and evaluate:

- `model.eval()` disables dropout so we get deterministic results
- We average loss over 20 batches for a stable estimate (single-batch loss is noisy)
- We check BOTH train and val loss

The gap between train and val loss tells us about overfitting. If train loss keeps dropping but val loss starts rising, the model is memorizing the training data instead of learning general patterns — like a student who memorizes answers instead of understanding concepts.

Section 7: Text Generation

Autoregressive Generation

python

```
@torch.no_grad()
def generate(model, prompt, max_new_tokens=500, temperature=0.8):
    model.eval()
    tokens = encode(prompt)
    tokens = torch.tensor(tokens).unsqueeze(0)

    for _ in range(max_new_tokens):
        context = tokens[:, -256:]          # Last 256 tokens only
        logits, _ = model(context)
        logits = logits[:, -1, :] / temperature # Last position only
        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, 1) # Sample from distribution
```

```
tokens = torch.cat([tokens, next_token], dim=1)
```

```
return decode(tokens[0].tolist())
```

This is how ChatGPT, Claude, and all LLMs generate text — one token at a time:

1. **Start with a prompt** (e.g., "ROMEO:")
2. **Feed it through the model** to get 65 scores for the next character
3. **Apply temperature** (divide logits by temperature)
4. **Convert to probabilities** (softmax)
5. **Sample** one character from the probability distribution
6. **Append** the new character to the sequence
7. **Repeat** from step 2

The **sliding window** (`tokens[:, -256:]`) ensures we never feed more than 256 tokens. If the generated text gets longer, we drop the oldest tokens — the model can only "remember" the last 256 characters.

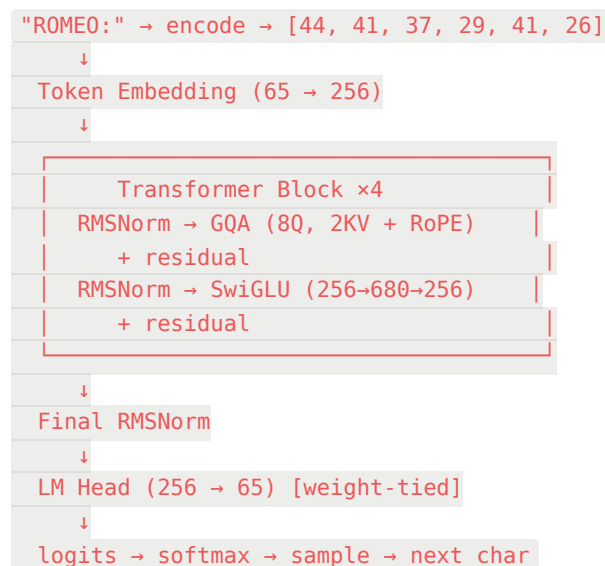
Temperature — Controlling Creativity

Temperature controls how "random" the sampling is:

Temperature	Behavior	Analogy
0.3 (low)	Very conservative, picks the most likely character almost every time. Repetitive but safe.	A student who always gives the textbook answer
0.8 (balanced)	Good mix of likely and surprising characters. Most natural output.	A student who usually follows the material but occasionally has creative insights
1.5 (high)	Very random, even unlikely characters get picked often. Creative but sometimes nonsensical.	A student who says wild things, sometimes brilliant, sometimes gibberish

How it works mathematically: Dividing logits by a small temperature makes the highest score relatively MUCH higher than the rest (sharper distribution). Dividing by a large temperature makes all scores more similar (flatter distribution).

The Complete Architecture — Summary



This is not a toy. Production LLMs like LLaMA 3, Mistral, and Gemma use these EXACT same components — RMSNorm, RoPE, GQA, SwiGLU. The only difference is scale: instead of 256 dimensions and 4 layers, they use 4096+ dimensions and 32+ layers.

Master Summary Table

Component	What It Replaces	What It Does	Why It's Better
RMSNorm	LayerNorm	Normalizes by dividing by RMS, then scaling	Simpler (2 ops vs 4), faster, same quality
RoPE	Sinusoidal PE	Rotates Q and K vectors by position-dependent angles	Captures relative position, not just absolute
GQA	Multi-Head Attention	Groups of Q heads share KV heads	4× smaller KV cache, much faster inference
SwiGLU	ReLU FFN	Dual-path with learnable gating	Gate learns what to keep/suppress, smoother activation
Pre-Norm	Post-Norm	Normalize BEFORE sub-layer	More stable training

Component	What It Replaces	What It Does	Why It's Better
Residual	Direct transform	Add sub-layer output to input	Easier gradient flow, learns "changes" not rebuilds
Weight Tying	Separate weights	Embedding and output head share weights	Fewer parameters, consistent token representations

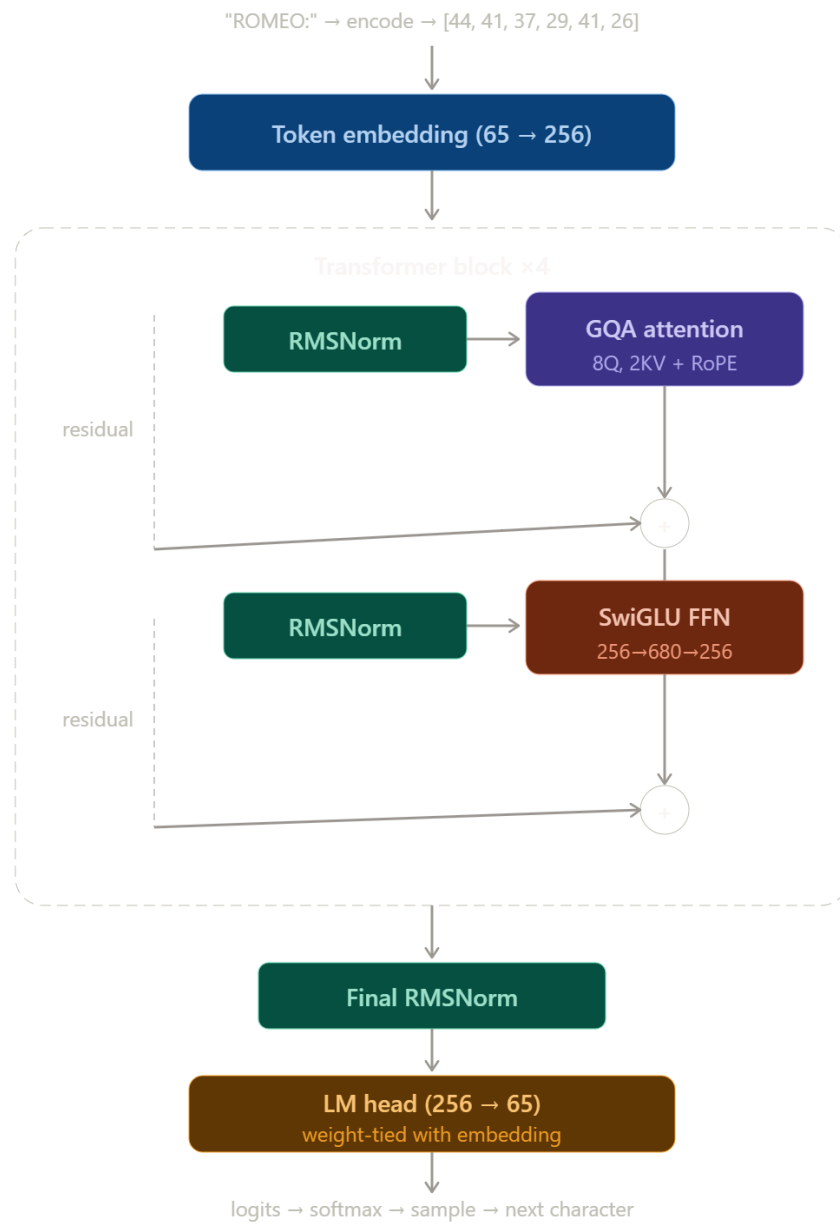
Experiments to Try

1. **Temperature sweep:** Generate "HAMLET: To be" at temperatures 0.1, 0.5, 0.8, 1.0, 1.5 — see how output changes from robotic repetition to creative chaos.
2. **Model surgery:** Print weight shapes of Q, K, V projections — see GQA's 4× difference in K/V vs Q sizes.
3. **Scaling question:** What happens if you double d_model from 256 to 512? (Hint: parameters roughly quadruple, training takes much longer, but quality improves.)

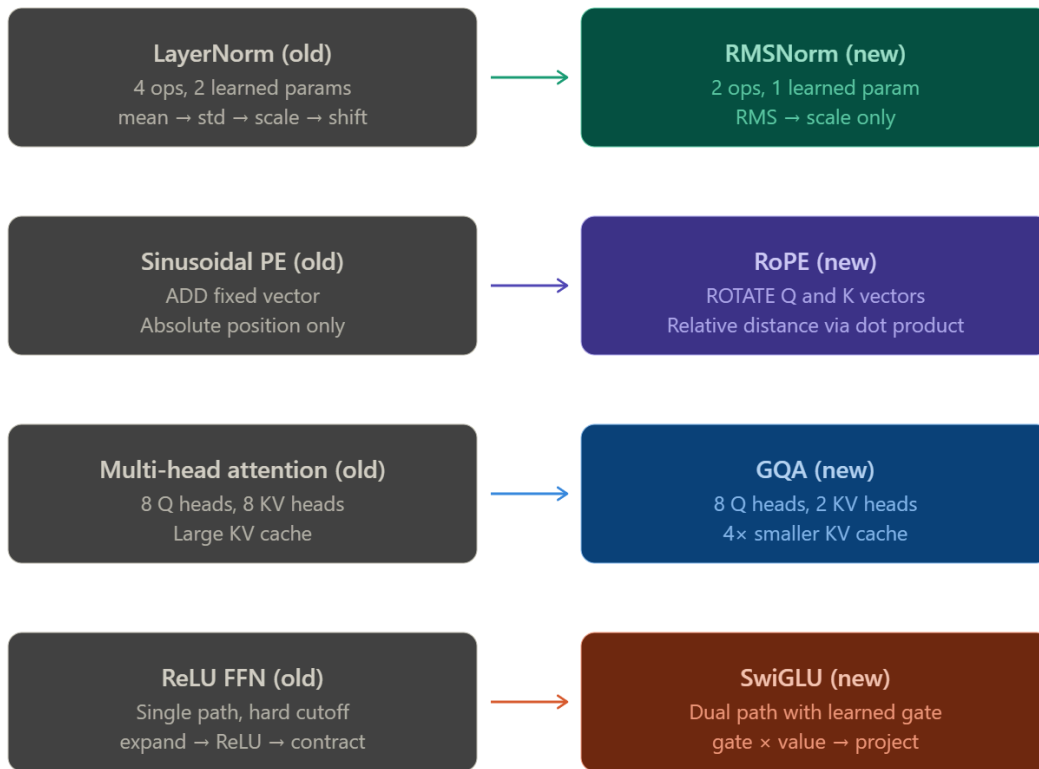
TODOs from the Lecture

- Train with more parameters for better quality
- Use different training data (not just Shakespeare)
- Scrape and train on other text (e.g., YouTube transcripts)
- Implement BPE tokenizer instead of character-level

Now let me create the key diagrams to accompany these notes:

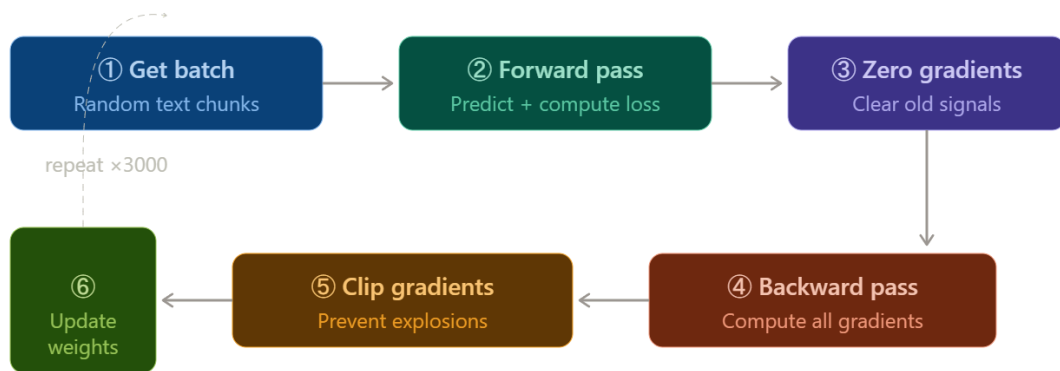


Here's the complete architecture showing how data flows through the model. Now let me show the key comparison between old and new components:



All four upgrades are used in production LLMs: LLaMA, Mistral, Gemma

Now let me show how attention and the training loop work:



And finally, let me show how autoregressive generation works:

